The Virtual Engineering Team

Running AI coding agents in parallel on one codebase

A coordination pattern we landed on.

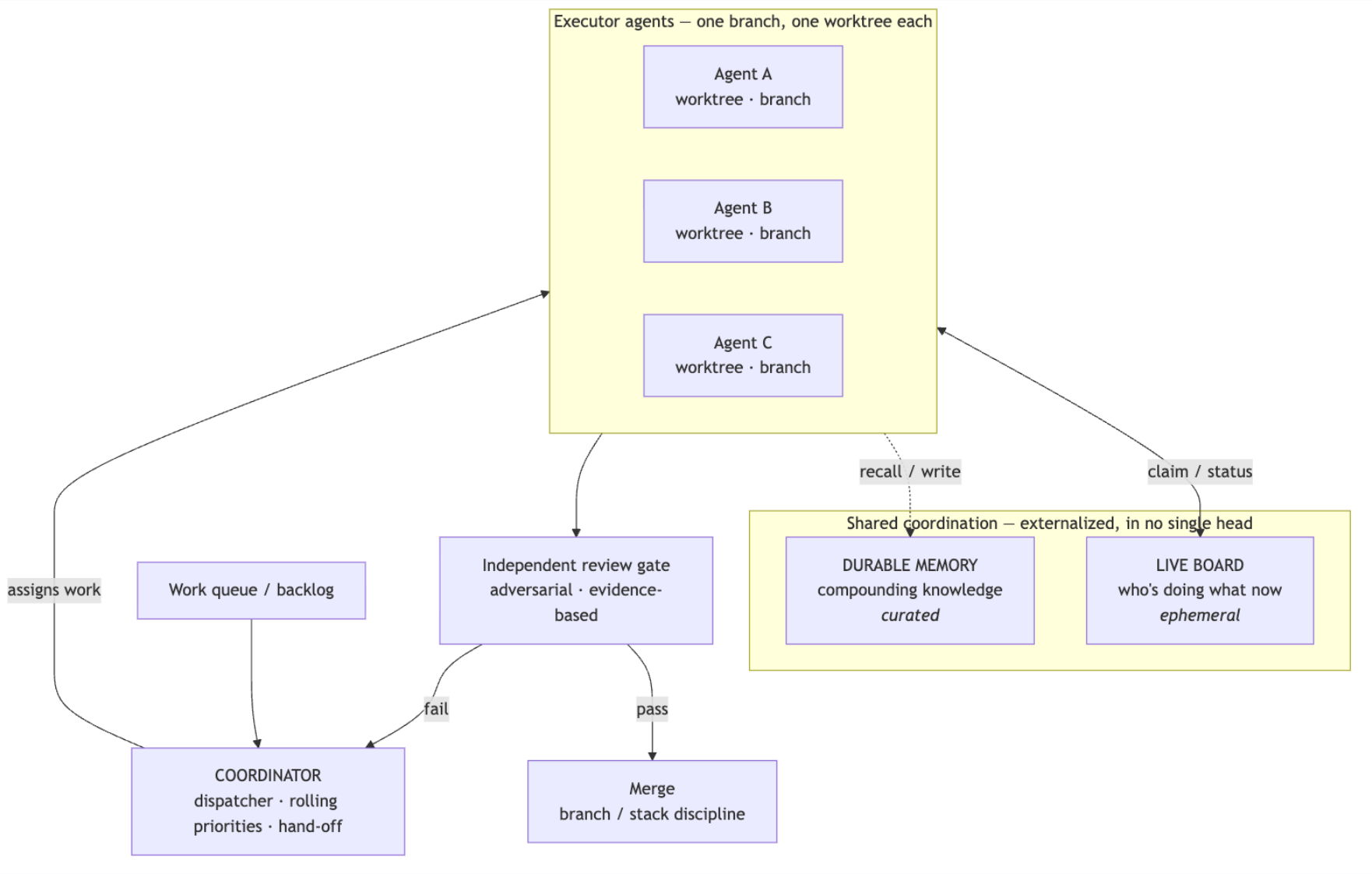
The problem

- One agent, one task, serial — wastes the thing that makes agents valuable: **you can run many at once.**
- Point several at the same codebase and they **collide**: two editing the same files, one clobbering another's branch, work duplicated, nothing reviewed, context lost.

> “ The question isn't "can an agent code?" It's **"how do you run a *team* of them without chaos?"** ”

The reframe

- N parallel agents face the **same problems a human engineering org already solved** — branching, code review, stand-ups, a tech lead, release hygiene.
- So don't invent new coordination tech. **Port the org chart and the SDLC onto agents** — and encode the rules where agents actually read them: repo, memory, a shared board.

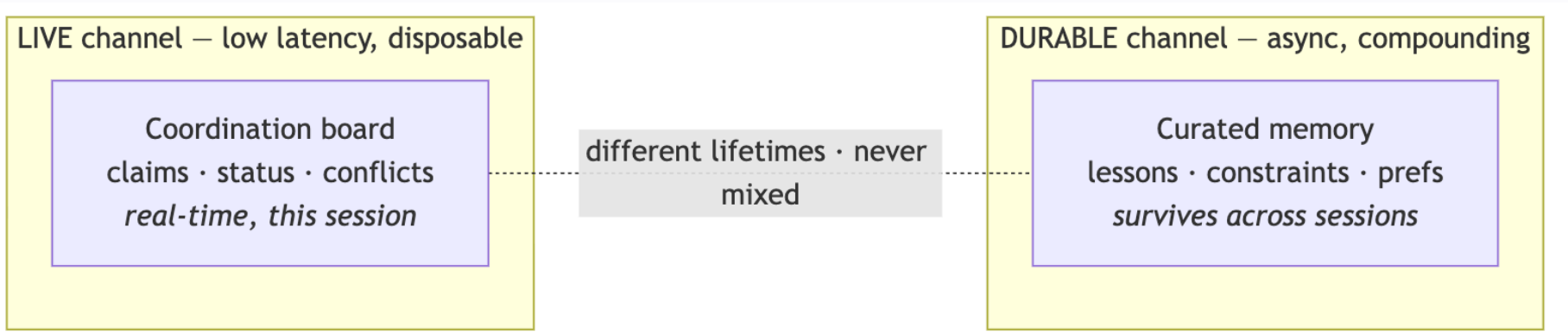

Executor agents — one branch, one worktree each
Agent A
worktree · branch
Agent B
worktree · branch
Agent C
worktree · branch
recall / write
claim / status
Shared coordination — externalized, in no single head
DURABLE MEMORY
compounding knowledge
curated
LIVE BOARD
who's doing what now
ephemeral
Independent review gate
adversarial · evidence-based
assigns work
Work queue / backlog
fail
pass
COORDINATOR
dispatcher · rolling priorities · hand-off
Merge
branch / stack discipline

Pillar 1 — Isolation: one branch, one workspace

- Every agent works in its **own git worktree** — never the shared checkout.
- The equivalent of feature branches + a private dev sandbox per engineer.
- Kills the *physical* collisions: file clobbering, shared build state, stash collisions.

Pillar 2 — Externalized coordination *(two channels)*

- Agents are **stateless** — nothing lives "in their head." Coordination must be **written down.**
- **Live board** *(ephemeral)* — who's doing what right now: claims, status, conflicts.
- **Durable memory** *(curated)* — lessons and constraints that compound across sessions.



Pillar 3 — The coordinator / dispatcher

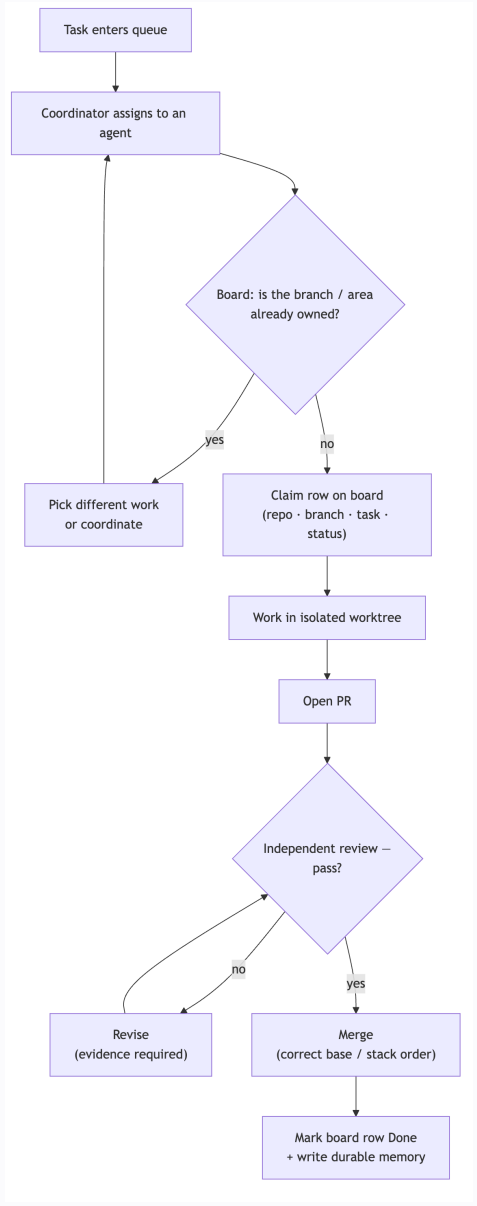
- One agent plays **tech-lead / PM**: owns the work queue, assigns tasks, keeps a **rolling priority list**, runs hand-off.
- Executors stay focused; priorities stay coherent; **nobody double-grabs** a task.

Pillar 4 — The claim protocol *(rules of engagement)*

Before starting, every agent:

- **Reads the board** — if someone owns the branch/area, pick different work or coordinate.
- **Claims** its row — repo · branch · task · status.
- **Marks done** when finished — releasing the claim.

> “Optimistic concurrency control for agents — lightweight advisory locks, not heavy gatekeeping.”


Task enters queue
Coordinator assigns to an agent
Board: is the branch / area already owned?
yes
no
Pick different work or coordinate
Claim row on board (repo · branch · task · status)
Work in isolated worktree
Open PR
Independent review — pass?
no
yes
Revise (evidence required)
Merge (correct base / stack order)
Mark board row Done + write durable memory

Pillar 5 — Adversarial, evidence-based review

- An agent can **confidently claim** something works when it doesn't.
- So review is **adversarial** — a different model or a fresh agent tries to *refute* the work.
- …and **evidence-based** — show the passing run, not an assertion.

> “ Code review + CI, adapted to the failure mode that's unique to agents. ”

Merge discipline

The boring release hygiene a human team needs — encoded so agents follow it:

- Branch off the **right base**.
- Correct **stacked-PR order** (don't orphan a stacked branch).
- **No force-push** after a PR merges.
- A **real build / bundle check** before merge — lint + unit tests miss integration breaks.

The two twists that make it *not* a human team

- **Stateless** → coordination must be **externalized** (no memory, no hallway chat).
- **Confident fabrication** → verification must be **adversarial + evidence-based** (no trust).

Everything else is **borrowed** from how humans already ship software.

What changes in practice

- **Real parallelism** without collision.
- **No lost or clobbered work** (isolation).
- **Coherent priorities**, no duplicated effort (coordinator + board).
- **Quality holds at speed** (adversarial review).
- **Knowledge compounds** across the whole fleet (durable memory).

> *Honest caveat:* coordination has overhead. It pays off only above a certain parallelism, and only if the protocol stays lightweight and the defaults are baked into the repo.

Why it generalizes

- It's an **org chart and an SDLC adapted for agents** — feature branches, a tech lead, code review, CI, a stand-up board.
- Portable to **any team running more than one AI engineer at a time**.

"An org chart for robots — happy to go deeper on the mechanics."